Sistema de Transporte Universitário

Documentação do modelo de dados (EDR)

Explica cada tabela, campo, relacionamento e regra do banco, como o sistema funciona no dia a dia e como montar e testar o banco no PostgreSQL com o DBeaver.

Conteúdo
1. Visão geral do sistema
2. Diagrama e como lê-lo
3. Tabelas, uma a uma
4. Enums (listas de valores)
5. Regras e travas do banco
6. Como o sistema funciona no dia a dia
7. A planilha do motorista, automática
8. Financeiro: mensalidades e salários
9. Relatórios prontos
10. Dados de teste
11. Como montar e testar no DBeaver
12. Decisões de modelagem

Banco de dados: PostgreSQL 15 ou superior · Diagrama: dbdiagram.io · Outubro de 2026

1. Visão geral do sistema

O sistema controla uma empresa de vans que leva estudantes de casa até a faculdade e de volta. Ele resolve quatro problemas:

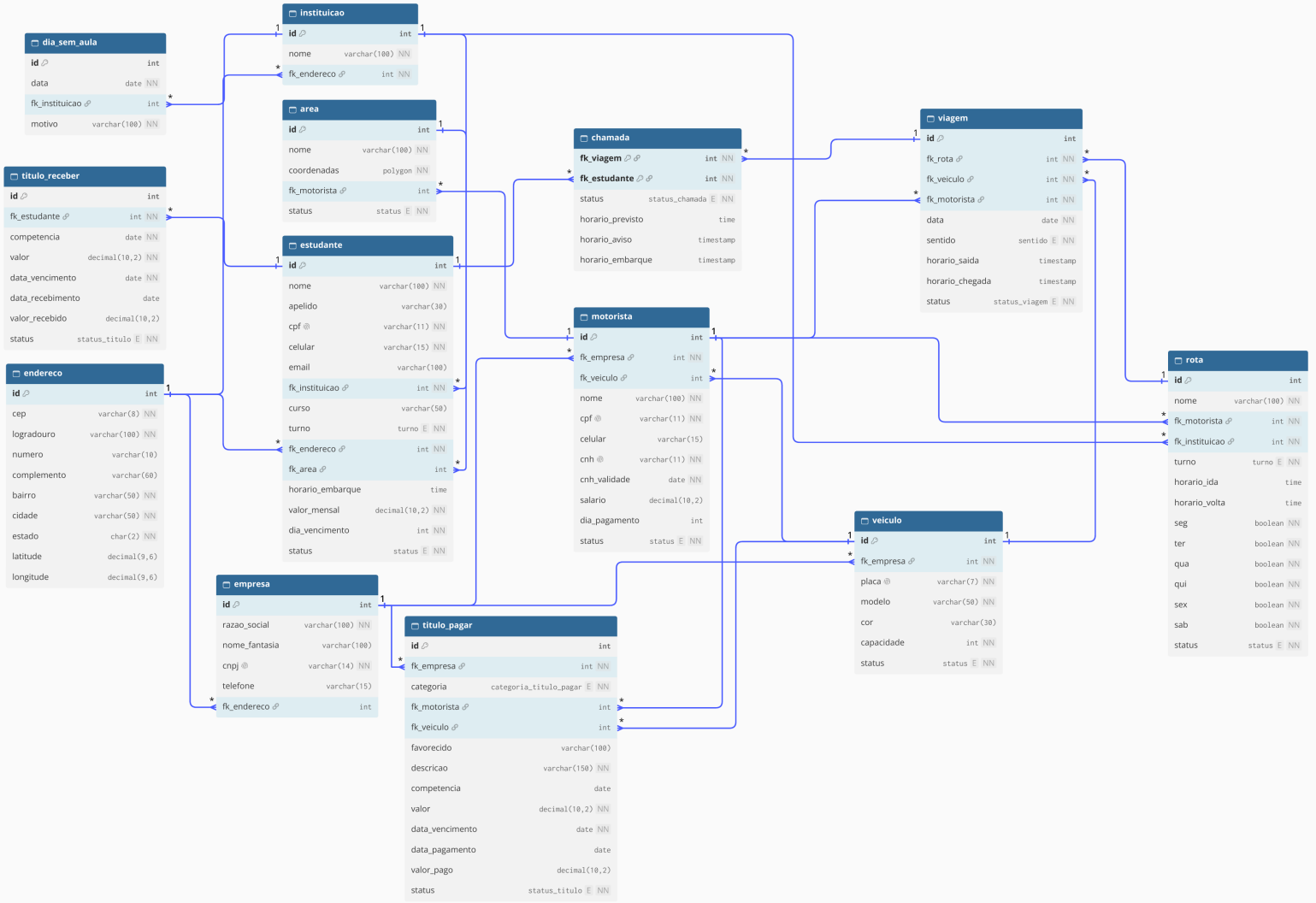
- **Quem cada motorista busca:** a cidade é dividida em **áreas** (bairros desenhados no mapa). Cada área pertence a um motorista. O aluno cai automaticamente na área do seu endereço.
- **Quem vai em cada viagem:** todo dia o sistema cria as viagens de ida e volta e a lista de alunos (**chamada**). Quem avisa que não vai sai da lista sozinho.
- **Quem embarcou:** o motorista marca presença no embarque, o que gera histórico de frequência.
- **Financeiro:** mensalidades dos alunos (**títulos a receber**) e salários e despesas (**títulos a pagar**).

Visão por grupos

Grupo	Tabelas	Função
Cadastros base	endereco, empresa, instituicao	Dados fixos que as outras tabelas usam.
Frota e equipe	veiculo, motorista	Vans e quem dirige.
Divisão da cidade	area	Bairros no mapa, cada um com seu motorista.
Clientes	estudante	Os alunos, onde moram, onde estudam e quanto pagam.
Planejamento	rota, dia_sem_aula	O trajeto fixo e os dias em que não há aula.
Operação diária	viagem, chamada	Cada ida/volta do dia e a presença de cada aluno.
Financeiro	titulo_receber, titulo_pagar	Entradas (mensalidades) e saídas (salários, despesas).

Resumo em uma linha
Empresa → tem veículos e motoristas → cada motorista cuida de áreas → os estudantes caem nas áreas pelo endereço → a rota liga o motorista a uma faculdade e turno → todo dia a rota gera viagens → cada viagem tem a chamada dos alunos. No financeiro, cada estudante gera uma mensalidade por mês e cada motorista um salário por mês.

2. Diagrama entidade-relacionamento



dbdiagram.io

Como ler o diagrama

Símbolo	Significado
Chave (ícone) / pk	**Chave primária**: identifica a linha de forma única. Em quase todas as tabelas é o `id`, um número que o banco gera sozinho (increment / serial).
fk_...	**Chave estrangeira**: guarda o id de outra tabela. Ex.: `motorista.fk_veiculo` guarda o id do veículo que ele usa.
1 ——— *	**Um para muitos**. O lado **1** é a tabela referenciada, o lado ***** é quem tem a fk. Ex.: uma instituição (1) tem muitos estudantes (*).
NN	**Not null**: campo obrigatório.
E	O tipo do campo é um **enum** (lista fechada de valores, ver seção 4).
Campo único (ícone)	**Unique**: não pode repetir. Ex.: dois motoristas não podem ter o mesmo CPF.

Relacionamentos principais

De	Para	Tipo	Significado
empresa	veiculo, motorista, titulo_pagar	1 : N	A empresa é dona da frota, da equipe e das despesas.
veiculo	motorista	1 : N	Veículo padrão do motorista. Uma van pode servir a mais de um motorista (turnos diferentes).
motorista	area	1 : N	Um motorista é responsável por várias áreas.
area	estudante	1 : N	Cada estudante fica em uma área (pelo endereço).
instituicao	estudante, rota	1 : N	Onde o aluno estuda / destino da rota.
motorista	rota	1 : N	Um motorista pode ter rotas em turnos ou faculdades diferentes.
rota	viagem	1 : N	Uma rota gera duas viagens por dia letivo (ida e volta).
viagem + estudante	chamada	N : N	A chamada liga alunos a viagens: um aluno em muitas viagens, uma viagem com muitos alunos.
estudante	titulo_receber	1 : N	Uma mensalidade por mês.
motorista / veiculo	titulo_pagar	1 : N	Salários do motorista e despesas do veículo.
endereco	empresa, instituicao, estudante	1 : N	Endereços ficam numa tabela só e são reaproveitados.

3. Tabelas, uma a uma

endereco

Guarda todos os endereços do sistema num lugar só. Empresa, faculdade e estudantes apontam para cá em vez de repetir rua, bairro e cidade em cada tabela.

Campo	Tipo	Para que serve
`id`	`int (pk)`	Identificador.
`cep`	`varchar(8)`	Só os 8 números. É texto porque CEP começa com zero (01310000) e int perderia o zero.
`logradouro`	`varchar(100)`	Rua, avenida.
`numero`	`varchar(10)`	Texto porque existem "S/N", "12A", "km 30".
`complemento`	`varchar(60)`	Apto, bloco, casa (opcional).
`bairro, cidade`	`varchar(50)`	Obrigatórios.
`estado`	`char(2)`	Sigla da UF (ES, SP...). A validação fica no formulário ou via CEP.
`latitude, longitude`	`decimal(9,6)`	Posição no mapa. É com ela que o banco descobre em qual **área** o endereço cai. Pode ser preenchida pela API de geolocalização ao cadastrar o CEP.

empresa

A transportadora dona das vans. Hoje existe uma, mas a estrutura aceita várias.

Campo	Tipo	Para que serve
`id`	`int (pk)`	Identificador.
`razao_social`	`varchar(100)`	Nome jurídico.
`nome_fantasia`	`varchar(100)`	Nome comercial.
`cnpj`	`varchar(14)`	Único. Só números (validado no banco).
`telefone`	`varchar(15)`	Contato.
`fk_endereco`	`int (fk)`	Endereço da sede.

veiculo

As vans e micro-ônibus da frota.

Campo	Tipo	Para que serve
`id`	`int (pk)`	Identificador.
`fk_empresa`	`int (fk)`	Dona do veículo.
`placa`	`varchar(7)`	Única, sem traço ou espaço (KZV9G47). O banco só aceita o formato antigo ou Mercosul.
`modelo`	`varchar(50)`	Van, Micro-ônibus, Sprinter...
`cor`	`varchar(30)`	Aparece na mensagem para o aluno reconhecer a van ("Van prata").
`capacidade`	`int`	Quantidade de lugares. Serve para checar se a rota não passou do limite.
`status`	`status`	ativo / inativo. Van vendida ou parada fica inativa sem perder o histórico.

motorista

Quem dirige. Também é a base para gerar o salário de cada mês.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
fk_empresa	int (fk)	Empresa onde trabalha.
fk_veiculo	int (fk)	Veículo que usa **normalmente**. No dia, a viagem pode usar outro.
nome, cpf, celular	varchar	Dados pessoais. CPF único, só números.
cnh, cnh_validade	varchar / date	Habilitação. A validade permite avisar antes de vencer.
salario	decimal(10,2)	Valor fixo mensal. A rotina mensal usa para criar o título a pagar.
dia_pagamento	int	Dia do mês seguinte em que o salário vence (padrão 5).
status	status	ativo / inativo.

area

Um bairro ou região desenhado no mapa, igual às áreas de entrega de um app de delivery. É o que define **quais alunos cada motorista busca**.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
nome	varchar(100)	Praia do Morro, Muquiçaba, Centro...
coordenadas	polygon	O desenho da área no mapa (lista de pontos longitude, latitude).
fk_motorista	int (fk)	Motorista responsável pela área.
status	status	Área desativada deixa de ser usada para novos cadastros.

Se uma área tiver alunos demais para uma van, o caminho é dividi-la em áreas menores e dar cada parte a um motorista.

instituicao

A faculdade de destino.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
nome	varchar(100)	Nome da faculdade.
fk_endereco	int (fk)	Onde fica.

estudante

O cliente: o aluno universitário, que é maior de idade e paga a própria mensalidade.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
nome	varchar(100)	Nome completo.
apelido	varchar(30)	Como aparece na lista do motorista: "Julia G", "Vinicius C". Diferencia alunos de mesmo nome.
cpf, celular, email	varchar	Contato. CPF único.
fk_instituicao	int (fk)	Onde estuda.
curso	varchar(50)	Informativo.
turno	turno	manhã, tarde ou noite. Define em qual rota ele entra.
fk_endereco	int (fk)	Ponto de embarque.

Campo	Tipo	Para que serve
fk_area	int (fk)	**Preenchido sozinho** pelo banco: ponto do endereço dentro do polígono da área. Fica vazio se o endereço não cair em nenhuma área (o gestor precisa resolver).
horario_embarque	time	Horário em que a van passa no ponto dele na ida (ex.: 17:25).
valor_mensal	decimal(10,2)	Quanto paga por mês. A rotina mensal usa para gerar a mensalidade.
dia_vencimento	int	Dia de vencimento da mensalidade (1 a 28, para existir em todo mês).
status	status	Inativo (trancou, saiu) não entra mais nas viagens, mas o histórico fica.

rota

O trajeto fixo: um motorista levando alunos para uma faculdade num turno. **Os alunos da rota não são cadastrados à mão**: são os estudantes das áreas daquele motorista, da mesma faculdade e do mesmo turno.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
nome	varchar(100)	Ex.: "Nilson - Noite - Faculdade de Vila Velha".
fk_motorista	int (fk)	Motorista titular.
fk_instituicao	int (fk)	Faculdade de destino.
turno	turno	Turno atendido.
horario_ida, horario_volta	time	Horário da primeira busca e da saída da faculdade.
seg ... sab	boolean	Em quais dias da semana a rota roda. Padrão: segunda a sexta.
status	status	Rota inativa não gera viagens.

Trava: um motorista não pode ter duas rotas para a mesma faculdade no mesmo turno.

dia_sem_aula

Feriados, recessos e férias. A rotina diária consulta esta tabela antes de criar viagens.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
data	date	O dia sem aula.
fk_instituicao	int (fk)	**Vazio** = vale para todas (feriado nacional). **Preenchido** = só aquela faculdade.
motivo	varchar(100)	Ex.: Finados, Recesso acadêmico.

viagem

Cada ida ou volta realizada num dia. É gerada automaticamente a partir da rota.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
fk_rota	int (fk)	De qual rota é.
fk_veiculo	int (fk)	Veículo **usado de fato** nesse dia. Começa com o padrão do motorista.
fk_motorista	int (fk)	Motorista **de fato**. Se o titular faltar, troca só aqui.

Campo	Tipo	Para que serve
data	date	Dia da viagem.
sentido	sentido	ida ou volta.
horario_saida, horario_chegada	timestamp	Gravados quando o motorista inicia e finaliza a viagem.
status	status_viagem	agendada → em_andamento → concluida (ou cancelada).

Trava: uma rota só tem uma viagem de ida e uma de volta por dia.

chamada

A lista de alunos de cada viagem: é a "planilha" do motorista. Cada linha é **um aluno em uma viagem**. A chave é a dupla viagem + estudante, então não há como duplicar.

Campo	Tipo	Para que serve
fk_viagem	int (pk, fk)	A viagem.
fk_estudante	int (pk, fk)	O aluno.
status	status_chamada	pendente, nao_vai, presente ou ausente (ver seção 4).
horario_previsto	time	Copiado do horário de embarque do aluno ao gerar. Pode ser ajustado só nesse dia.
horario_aviso	timestamp	Quando o aluno avisou que não vai (comprova se avisou no prazo).
horario_embarque	timestamp	Quando embarcou de fato.

titulo_receber

Contas a receber: as mensalidades dos alunos.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
fk_estudante	int (fk)	Quem paga.
competencia	date	Mês de referência, sempre dia 01 (2026-10-01 = outubro).
valor	decimal(10,2)	Valor cobrado.
data_vencimento	date	Quando vence.
data_recebimento, valor_recebido	date / decimal	Preenchidos quando o aluno paga. O valor pode diferir (desconto, juros).
status	status_titulo	aberto, pago ou cancelado.

Trava: uma mensalidade por aluno por mês. **Atrasado não é um status gravado**: é título aberto com vencimento menor que hoje.

titulo_pagar

Contas a pagar: salários dos motoristas e despesas da operação.

Campo	Tipo	Para que serve
id	int (pk)	Identificador.
fk_empresa	int (fk)	Quem paga.
categoria	enum	salario, combustivel, manutencao, seguro, impostos, outros.
fk_motorista	int (fk)	Preenchido quando é **salário** (obrigatório nesse caso).
fk_veiculo	int (fk)	Preenchido em combustível, manutenção, seguro de um veículo. Permite saber o custo de cada van.

Campo	Tipo	Para que serve
favorecido	varchar(100)	Quem recebe quando não é motorista: posto, oficina, seguradora.
descricao	varchar(150)	Texto livre.
competencia	date	Mês a que a despesa pertence (dia 01).
valor	decimal(10,2)	Valor previsto.
data_vencimento	date	Quando vence.
data_pagamento, valor_pago	date / decimal	Preenchidos quando é pago. Hora extra ou desconto entram no valor_pago.
status	status_titulo	aberto, pago ou cancelado.

Travas: salário sem motorista é recusado, e só existe um salário por motorista por mês.

4. Enums (listas de valores)

Enum é um tipo que só aceita os valores da lista. Evita erros de digitação ("Ativo", "ATIVO", "ativ") e deixa os dados legíveis, ao contrário de usar números (1 = ativo, 2 = inativo).

Enum	Valores	Usado em
status	ativo, inativo	veiculo, motorista, area, estudante, rota
turno	manha, tarde, noite	estudante, rota
sentido	ida, volta	viagem
status_viagem	agendada, em_andamento, concluida, cancelada	viagem
status_chamada	pendente, nao_vai, presente, ausente	chamada
status_titulo	aberto, pago, cancelado	titulo_receber, titulo_pagar
categoria_titulo_pagar	salario, combustivel, manutencao, seguro, impostos, outros	titulo_pagar

Os status da chamada

Status	Quando	Aparece na lista?
pendente	Criada de madrugada pelo sistema, aguardando a viagem.	Sim
nao_vai	O aluno avisou pelo app, dentro do prazo, que não vai.	Não (vai para "Não vão hoje")
presente	O motorista marcou o embarque.	Sim, com o horário real
ausente	Não avisou e não apareceu. A viagem encerra e o que ficou pendente vira ausente.	—

5. Regras e travas do banco

Além do diagrama, o script SQL coloca regras que impedem dados errados de entrarem, mesmo que o programa tenha um bug:

Regra	Onde	O que impede
Formato de CPF, CNPJ, CEP	CHECK	Letras, pontos, tamanho errado.
Formato de placa	CHECK	Placa inválida (aceita ABC1234 e Mercosul ABC1D23).
Dia de vencimento / pagamento entre 1 e 28	CHECK	Dia 31 em fevereiro.
Valores não negativos	CHECK	Mensalidade, salário, capacidade negativos.
Competência sempre dia 01	CHECK	Dois jeitos de representar o mesmo mês.
CPF, CNPJ, CNH, placa únicos	UNIQUE	Cadastro duplicado.
Uma rota por motorista + faculdade + turno	UNIQUE	Rotas repetidas.
Uma viagem por rota + data + sentido	UNIQUE	Gerar o dia duas vezes.
Um aluno por viagem	PK composta	Aluno duplicado na lista.
Uma mensalidade por aluno por mês	UNIQUE	Cobrar duas vezes.
Um salário por motorista por mês	Índice único parcial	Pagar salário duas vezes.
Salário exige motorista	CHECK	Título de salário solto.
Um feriado por data + faculdade	UNIQUE NULLS NOT DISTINCT	Feriado lançado duas vezes.

6. Como o sistema funciona no dia a dia

O banco só guarda os dados. Quem cria as viagens e as cobranças são **rotinas automáticas** (funções no banco, chamadas por uma tarefa agendada do backend, um cron).

Madrugada: gerar_dia(data)

Roda todo dia, por exemplo às 5h:

- Para cada rota ativa, verifica se ela roda naquele **dia da semana** (colunas seg a sab) e se o dia **não está em dia_sem_aula** (para todas ou para aquela faculdade).
- Cria duas **viagens** (ida e volta) com o motorista e o veículo padrão.
- Para cada viagem, busca os estudantes **ativos** das **áreas do motorista da rota**, da mesma faculdade e turno, e cria a **chamada** de cada um como pendente, copiando o horário de embarque.
- Pode rodar mais de uma vez sem duplicar nada.

```
SELECT transporte.gerar_dia(current_date);
```

Durante o dia

Quem	Faz o quê	O que muda no banco
Aluno	Avisa pelo app que não vai.	Chamada vira **nao_vai**, grava horario_aviso. Sai da lista na hora.
Gestor	Motorista faltou / van quebrou.	Troca fk_motorista ou fk_veiculo **só na viagem do dia**. O cadastro não muda.
Motorista	Inicia a viagem.	Viagem vira **em_andamento**, grava horario_saida.
Motorista	Toca no nome do aluno no embarque.	Chamada vira **presente**, grava horario_embarque.
Motorista	Finaliza a viagem.	Viagem vira **concluida**; quem ficou pendente vira **ausente**.

Aviso de "não vou": avisar_nao_vou

O aluno só pode avisar até **1 hora antes** do horário dele. Depois disso, se não aparecer, conta como ausente. A função devolve **OK** ou o motivo da recusa.

```
SELECT avisar_nao_vou(10, '2026-10-26', 'ida');
-- OK
-- ou: Fora do prazo: avisar até 1h antes de 17:25
```

Se o aluno se arrepender dentro do prazo, o sistema volta a chamada para pendente e ele reaparece.

Fim da viagem: concluir_viagem(id)

Marca a viagem como concluída e transforma em **ausente** todo aluno que ficou pendente.

Volume de dados

Com 10 rotas e 300 alunos: 20 viagens e cerca de 600 chamadas por dia, uns 150 mil registros por ano. Para o PostgreSQL isso é muito pouco.

7. A planilha do motorista, automática

Hoje a planilha é montada à mão e o nome de quem não vai é apagado. No sistema, cada parte da mensagem já está no banco e a função `texto_planilha(rota, data, sentido)` monta o texto pronto:

Na mensagem	Vem de
"Segunda-feira"	viagem.data

Na mensagem	Vem de
"Com Nilson"	motorista.nome (o da viagem: pega o substituto se houver)
"Van prata placa KZV 9G47"	veiculo.modelo, cor e placa (da viagem)
Nomes e horários	chamada + estudante.apelido, sem quem está nao_vai, ordenado por horário
"Não vão hoje"	chamada com status nao_vai

Comando

```
SET search_path
  TO transporte;

SELECT texto_planilha(
  1, '2026-10-26');
```

Resultado (dados de teste)

```
Boa tarde
Planilha dessa Segunda-feira
Com Nilson
Van prata placa KZV 9G47

Gabriella...17:25
Harley......17:25
Julia G.....17:25
Guilherme...17:35
Nicolas.....17:35
Roberto.....17:35
Vinicius C..17:35
Vinicius G..17:35
Gabriel.....17:40
Katherine...17:45
Victoria....17:45
Junior......18:05

Não vão hoje: Lucas, Mariana
```

Na prática
O sistema mostra o texto com um botão "Copiar para WhatsApp", e no futuro pode enviar sozinho no grupo pela API do WhatsApp. No app do motorista, a mesma lista vira o checklist de embarque. Nada é apagado: quem não vai continua registrado, e isso alimenta o relatório de frequência.

8. Financeiro: mensalidades e salários

Rotina mensal: gerar_titulos_mes(competência)

Roda no dia 1º de cada mês e cria, de uma vez:

- **Uma mensalidade por aluno ativo** (titulo_receber), com o valor_mensal dele e vencimento no dia_vencimento do próprio mês.
- **Um salário por motorista ativo** (titulo_pagar, categoria salario), com o valor do campo salario e vencimento no dia_pagamento do mês seguinte.
- Pode rodar mais de uma vez: o que já existe não é duplicado.

```
SELECT gerar_titulos_mes('2026-10-01');
```

Exemplo: o salário do Nilson

Etapa	O que acontece
Cadastro	motorista.salario = 3.200,00 · dia_pagamento = 5
01/10	Rotina cria o título: categoria salario, fk_motorista = Nilson, competência 10/2026, valor 3.200,00, vencimento 05/11/2026.
05/11	Empresa paga. Preenche data_pagamento e valor_pago (3.200,00, ou outro valor se teve hora extra ou desconto) e status = pago.

Por que título a pagar e não "despesa"

- Segue o mesmo padrão das mensalidades (título a receber), então os dois lados do caixa funcionam igual.
- Separa o **previsto** (valor, vencimento) do **realizado** (valor_pago, data_pagamento).
- Com a **competência**, o salário de outubro conta em outubro, mesmo sendo pago em novembro. Isso dá o resultado real de cada mês.
- Ligado ao motorista e ao veículo, permite saber o custo de cada um.

9. Relatórios prontos (views)

Consulta	Mostra
SELECT * FROM vw_frequencia;	Por aluno: presenças, avisos de não vou, faltas sem aviso e % de presença.
SELECT * FROM vw_inadimplentes;	Mensalidades em aberto e vencidas, com celular e dias de atraso.
SELECT * FROM vw_resultado_mes;	Por mês: receitas (mensalidades), despesas (títulos a pagar) e resultado.

Outras perguntas que o modelo responde com uma consulta simples: quem está na van agora, quem faltou hoje, quantos alunos cada motorista tem versus a capacidade da van, CNHs vencendo nos próximos 30 dias, custo de combustível por veículo, pontualidade de cada motorista (horário previsto x real).

10. Dados de teste

O arquivo `02_dados_teste.sql` simula a operação em Guarapari (ES) levando alunos a uma faculdade em Vila Velha, turno da noite. O "hoje" do teste é **segunda, 26/10/2026**.

O que tem	O que demonstra
2 motoristas: Nilson (van prata KZV9G47) e Marcos (van branca)	Motorista com veículo padrão.
3 áreas: Praia do Morro e Muquiçaba (Nilson), Centro (Marcos)	Divisão da cidade por motorista.
14 alunos do Nilson, 4 do Marcos	Área preenchida sozinha pelo endereço.
Aluna do turno da manhã	Não entra na rota da noite.

O que tem	O que demonstra
Aluno com endereço fora das áreas	Fica sem área, para o gestor resolver.
Aluno inativo	Sai das próximas viagens.
Viagens de 13/10 a 26/10	Feriado de 12/10 e fins de semana não geram viagens.
Dia 28/10 sem aula só na faculdade	Feriado por instituição.
21/10: van do Nilson quebrou	A volta foi feita pelo Marcos no micro-ônibus, sem mudar cadastros.
Lucas e Mariana avisaram que não vão hoje	Saem da lista; a planilha fica igual à real.
Harley tentou avisar às 17h	Recusado por estar fora do prazo.
Setembro pago; outubro com 3 alunos em atraso	Inadimplência e resultado do mês.
Combustível, troca de embreagem e seguro	Despesas por veículo e por categoria.

11. Como montar e testar no DBeaver

#	Passo	Como
1	Ligar o PostgreSQL	No Windows, abra **Serviços** (Win + R → services.msc), dê dois cliques em **postgresql-x64-17**, coloque Tipo de inicialização **Automático**, Aplicar, Iniciar.
2	Conectar	DBeaver → Banco de dados → Nova conexão → PostgreSQL. Host localhost, porta do seu PostgreSQL (5432 ou 5433, veja `port =` no postgresql.conf), usuário postgres e sua senha.
3	Criar o banco	Editor SQL na conexão e rode `CREATE DATABASE transporte_universitario;` Aperte F5. Se não aparecer, marque "Exibir todos os bancos de dados" na conexão.
4	Abrir editor no banco novo	Botão direito em transporte_universitario → Editor SQL → Novo script. Confira o nome do banco no topo do editor.
5	Rodar a estrutura	Cole o 01_estrutura.sql e aperte **Alt + X** (executar script). Não use Ctrl + Enter, que roda só um comando e quebra as funções.
6	Rodar os dados	Cole o 02_dados_teste.sql e aperte Alt + X.
7	Testar	`SET search_path TO transporte;` e `SELECT texto_planilha(1, '2026-10-26');` Clique na célula e veja na aba Valor.

Esqueceu a senha do postgres?
1. Abra `C:\Program Files\PostgreSQL\17\data\pg_hba.conf` no Bloco de Notas **como administrador**.
2. Troque `scram-sha-256` por `trust` e salve.
3. Reinicie o serviço postgresql-x64-17.
4. Conecte com senha em branco e rode `ALTER USER postgres PASSWORD 'nova';`
5. Volte para `scram-sha-256` e reinicie o serviço. Não deixe em trust: qualquer programa entraria no banco sem senha.

Para ver o diagrama, cole o `transporte_universitario.dbml` em dbdiagram.io/d.

12. Decisões de modelagem

Decisão	Por quê
Sem tabela de responsável	Transporte universitário: o aluno é maior de idade e paga a própria mensalidade.
Sem tabela de contrato	Valor mensal e vencimento ficam no estudante. É a mesma informação com uma tabela a menos.
Alunos da rota calculados pela área	Ninguém precisa incluir aluno na rota à mão: o endereço define a área, a área define o motorista. Mudou de casa, mudou de rota sozinho.
Veículo no motorista, e não o contrário	A mesma van pode atender motoristas de turnos diferentes.
Motorista e veículo também na viagem	Substituições do dia não alteram cadastros e o histórico mostra quem rodou de fato.
Nada é apagado	Aluno que não vai vira nao_vai; aluno que sai vira inativo. O histórico de frequência e cobrança se mantém.
Atrasado é calculado	Se fosse gravado, alguém teria que atualizar todo dia. Aberto + vencido = atrasado.
Endereço em tabela própria	Evita repetir rua, bairro e cidade em várias tabelas e concentra a latitude/longitude.
Documentos como texto	CPF, CNPJ, CEP e celular podem começar com zero e não são usados em contas.
Dinheiro em decimal(10,2)	Float tem erro de arredondamento; decimal é exato.
Enums no lugar de números	status = 'pago' é legível; status = 2 exige decorar o significado.
PostgreSQL	Tem tipo polygon e o teste "ponto dentro da área" prontos, sem extensão.

Possíveis evoluções

- **Ordem de parada** manual por aluno, se o horário não bastar para ordenar a lista.
- **Endereço de embarque diferente na volta** (ex.: aluno que volta para o trabalho).
- **Área com mais de um motorista**: trocar fk_motorista por uma tabela motorista_area.
- **Usuários e login** para o app do aluno, do motorista e do gestor.
- **Geolocalização em tempo real** da van, para o aluno acompanhar a chegada.